

Spring AI in der Praxis

Spring AI Basics

Thorben Janssen

Independent consultant, trainer and author



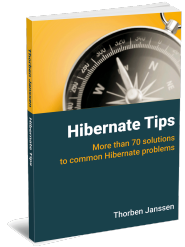


Co-Organizer

JUG Paderborn



thorben-janssen.com



hibernate-tips.com

Persistence Hub

persistence-hub.com

@thjanssen123

/c/ThoughtsOnJava

thorbenjanssenofficial

Agenda

- Spring AI basics
- Model evaluation
- RAG
- Tool calling & MCP

Schedule

- 09:00 - 10:30
- 11:00 - 12:30
- 13:30 - 15:00
- 15:30 – 17:00

Code

- https://github.com/thjanssen/SpringAi-EvalRagMcp
- OpenAI API key in chat
- Ollama users
  - ollama pull qwen3:8b
  - ollama pull bespoke-minicheck

Building your 1st ChatClient



Live Demo

Enhancing LLMs

- LLMs are stateless
- Available information
  - Training data
  - Information in prompt

Enhancing LLMs

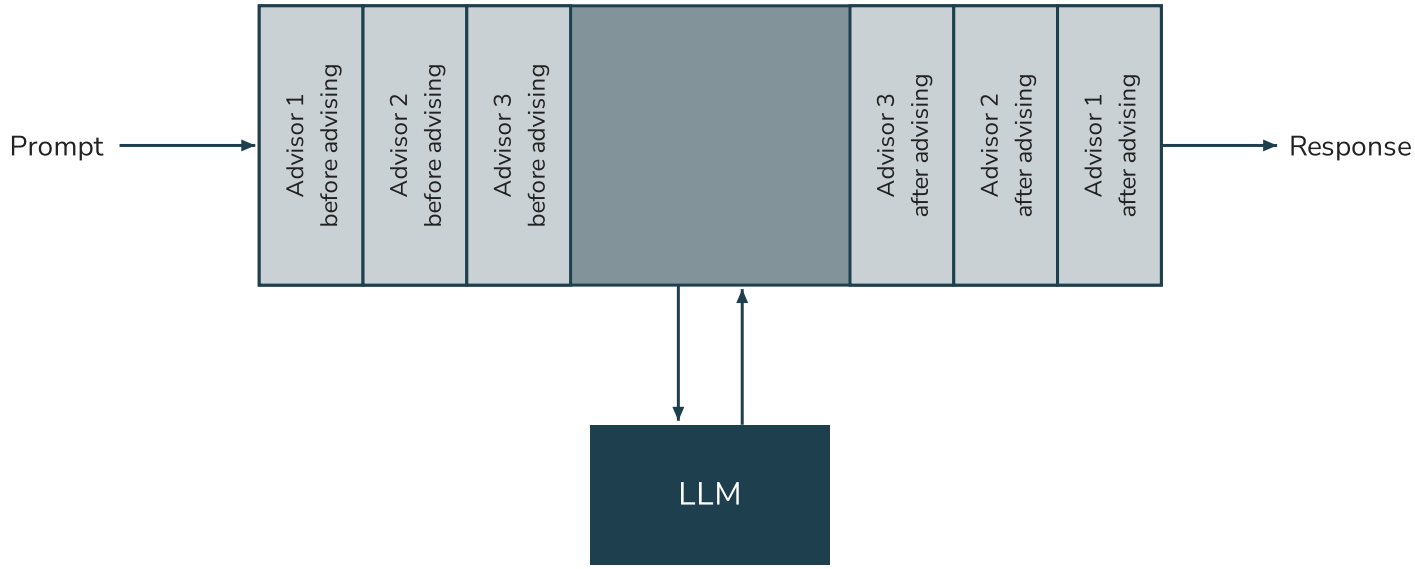
- Prompt Engineering
  - Structured output
  - Few-shot prompting
  - Prompt stuffing

Live Demo

Advisors

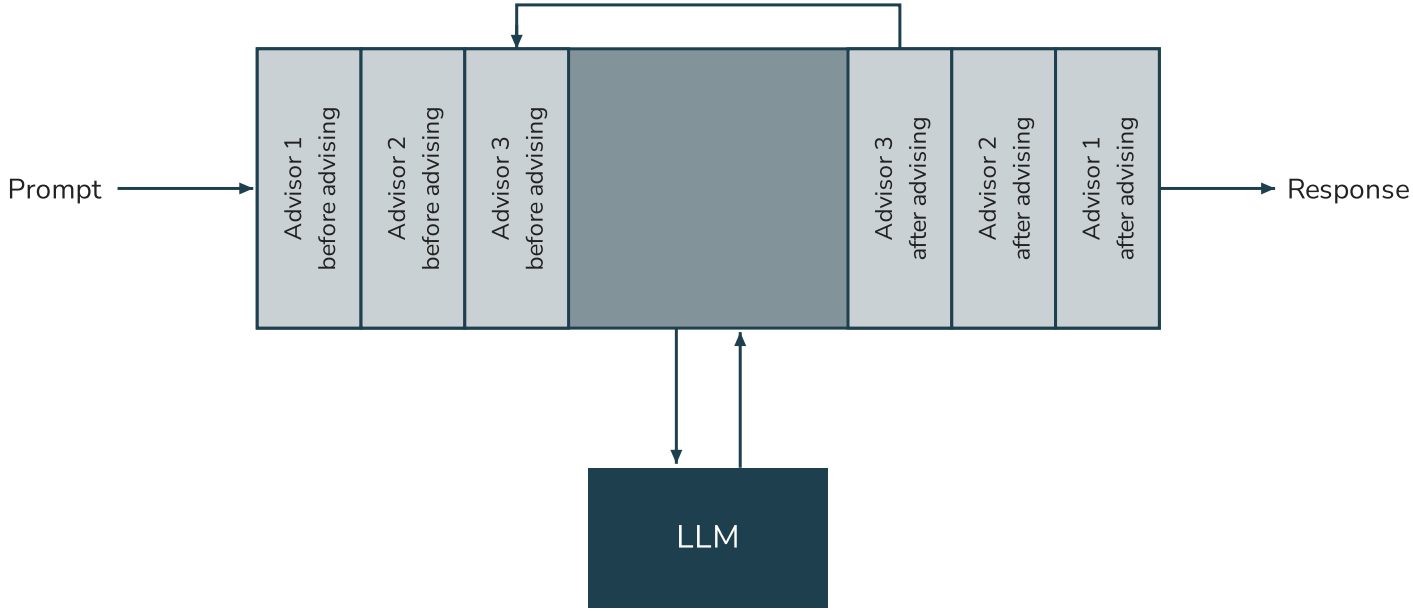


AdvisorChain



Live Demo

Recursive AdvisorChain



Live Demo

Session Memory



Conversation Context

- Problem
  - LLMs are stateless
  - Humans remember conversation‘s context
- Solution
  - Add context to each prompt

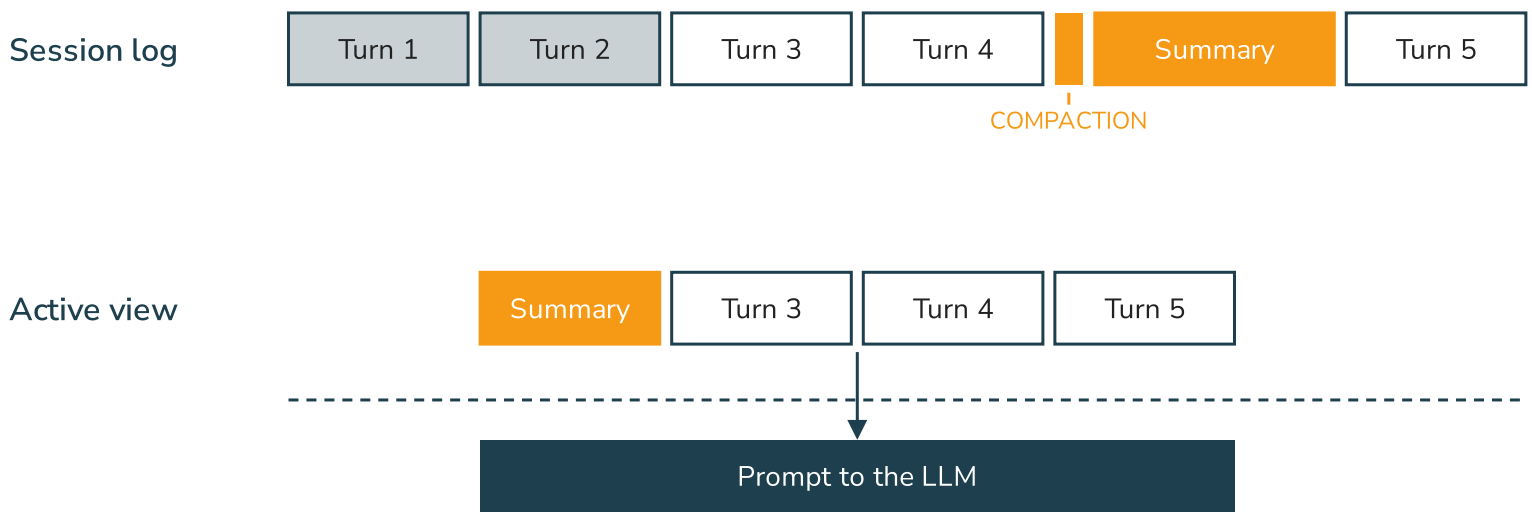
ChatMemory (until Spring AI 2.0)

- Window of messages per conversation
  - `MessageChatMemoryAdvisor` adds them to each prompt
- Problems
  - Message-based: request and response can get separated
  - Tool calls: one request = unknown number of messages
  - Older messages drop out of the window, with their information
- Deprecated in 2.1, removal in 3.0

Session Memory

- New in Spring AI 2.1
- Session log
  - Append-only event log of the conversation
  - Nothing gets removed or changed
- Active view
  - Subset of the log that gets added to the prompt
- Turn-aware
  - Turn = user request … final response, incl. all tool calls
- Compaction
  - Summarizes older turns

Session Log and Active View



TurnCountTrigger(3), WindowCompactionStrategy keeps the last 2 turns

Dependencies

- Starter with JDBC repository

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-model-session-jdbc</artifactId>
</dependency>
```

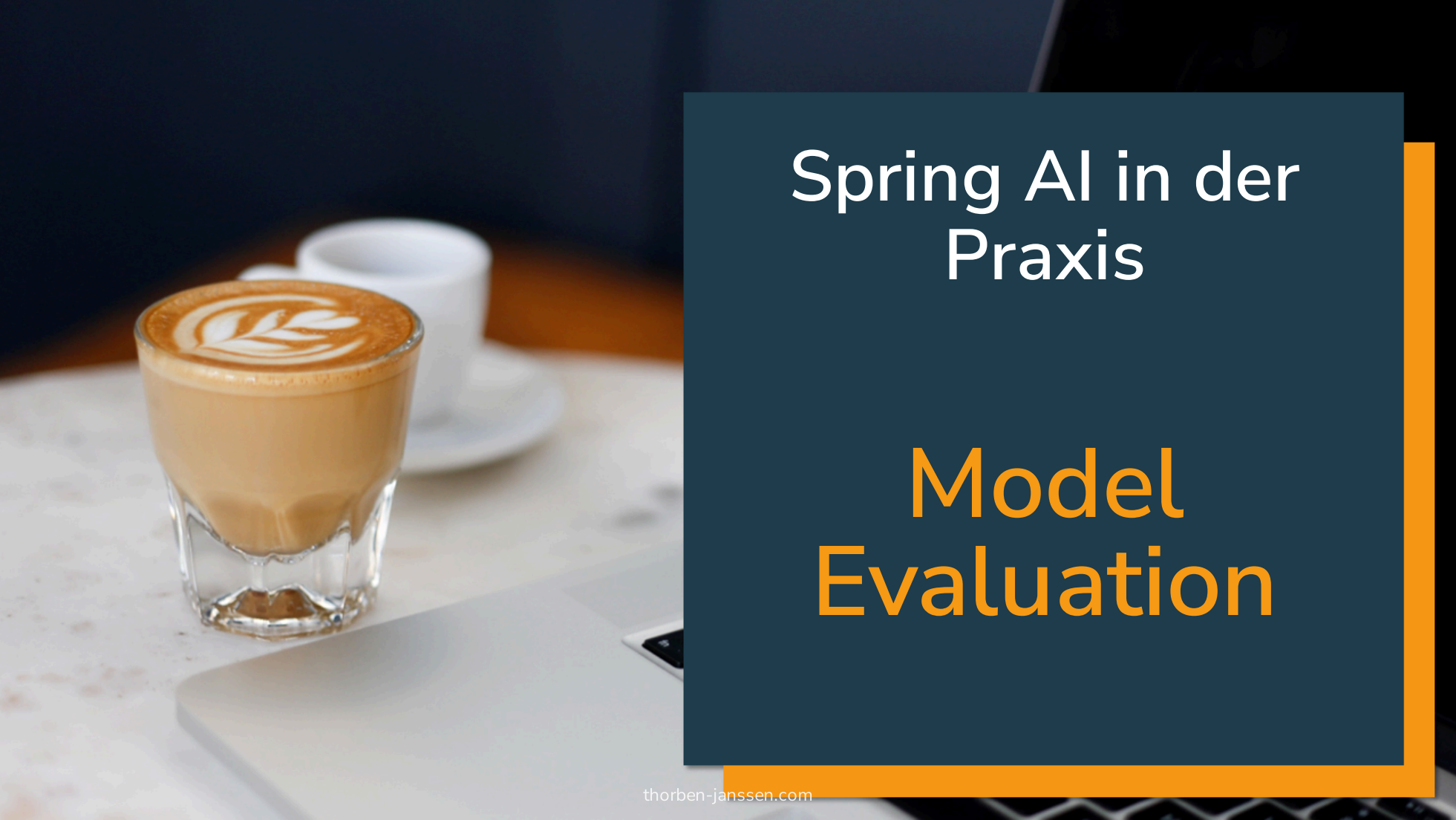
- Create the tables (default: only for in-memory databases)

```
spring.ai.session.repository.jdbc.initialize-schema: always
```

Live Demo

Questions





Spring AI in der Praxis

Model Evaluation

LLMs are non-deterministic



Problem

- Prompt: „What‘s the capital of Germany?“
  - „Berlin“
  - „Berlin is the capital of Germany.“
  - „Berlin was the capital of Germany but now it‘s Bonn.“

Problem

- No single correct answer
- Examples
  - Summarizing a text
  - Classifying a support request
  - Generating an email

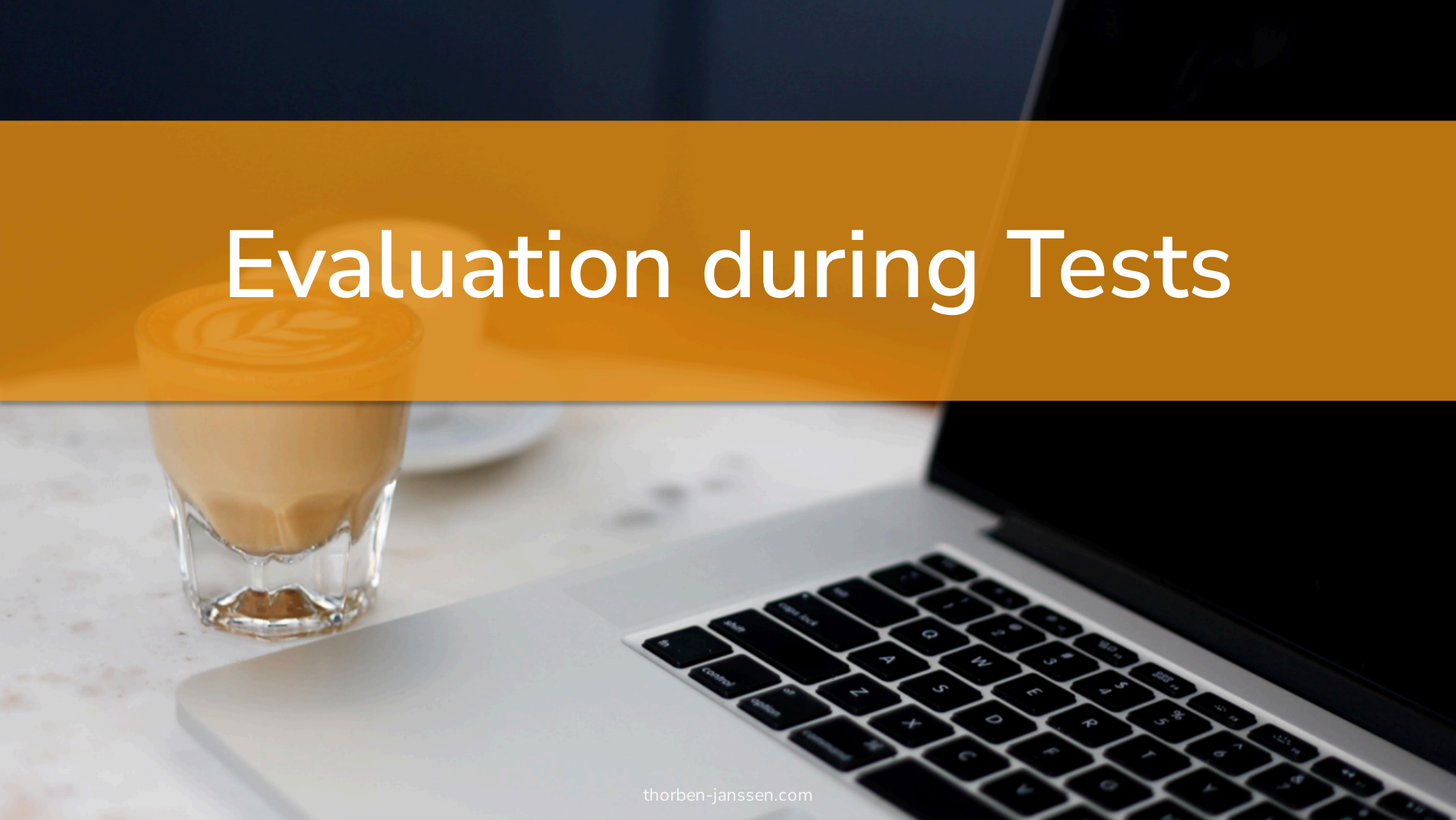
LLM as a Judge

- Ask LLM to evaluate a text
  - Requires special evaluation prompt
  - Use structured output
- Specialized models
  - Judge Arena: https://huggingface.co/spaces/AtlaAI/judge-arena

Typical Evaluations

- Correctness
- Relevancy
- Politeness
- Tone
- Company policies

Evaluation during Tests



Test Evaluation

- Based on `Evaluator` interface
- Standard `Evaluator`s
  - `FactCheckingEvaluator`
  - `RelevancyEvaluator`

Live Demo

Local Models

- Problem
  - Limited resources
  - Weak models

Local Models

- Options
  - Local inference server
  - Testcontainers
    - Docker Model Runner
    - Ollama in a Docker container

Live Demo

Costs & Complexity



Assertions

- Structured output
- Separate deterministic from non-deterministic
  - Business logic
  - Some Advisors
  - Some Tools
  - RAG retrieval

Test Model

- Use test models
  - Technical end-to-end tests
  - Advisor chains
  - RAG

Final Tests

- Always test like production
  - Same model
  - Same vector store
  - Same tools / MCP servers

Evaluation at Runtime



Runtime Evaluation

- Constraints
  - Time
  - Money / resources

Runtime Evaluation

- Based on
  - Recursive `Evaluator`
  - Support for multiple `ChatModel`s

Live Demo

Runtime Evaluation

- https://huggingface.co/learn/cookbook/en/llm_judge#3-improve-the-llm-judge
- https://spring.io/blog/2025/11/10/spring-ai-llm-as-judge-blog-post

Spring Community: Agent Judge



Agent Judge

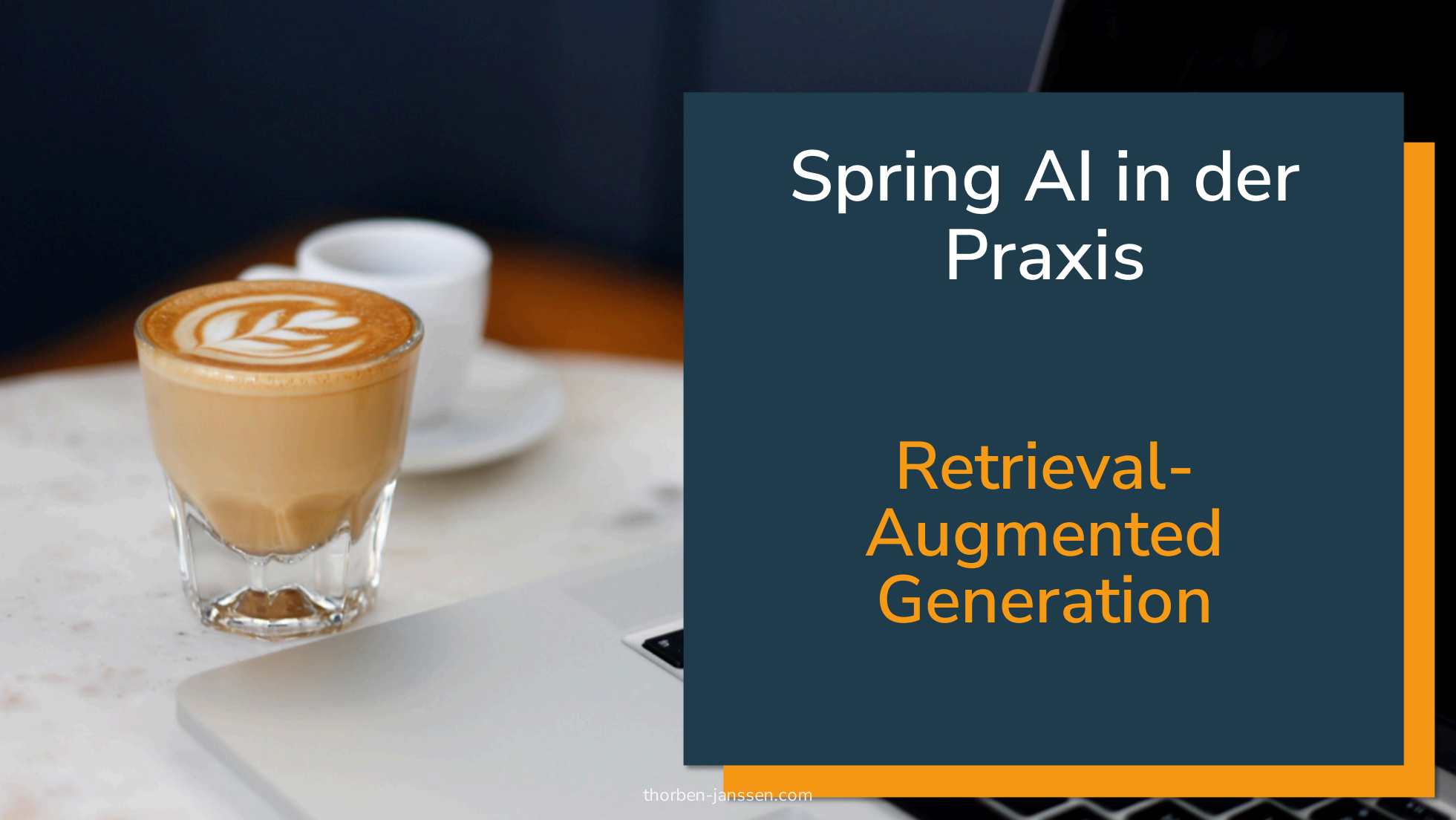
- Agent-agnostic evaluation framework
- Judges based on
  - Command execution
  - LLM
  - File comparison
- Supports judge composition

Agent Judge

- https://github.com/spring-ai-community/agent-judge

Questions





Spring AI in der Praxis

Retrieval-Augmented Generation

RAG

- Provide additional context / data to your LLM
- Goal
  - Provide custom information
  - Provide current information
  - Reduce hallucination

RAG

- Preparation
  - Store information in vector store
- Application workflow
  - Query vector store for relevant information
  - Add information to prompt

Dependencies

- Out-of-the-box support for RAG
  - spring-ai-advisors-vector-store
- Vector stores
  - spring-ai-starter-vector-store-<VectorStoreName>

Live Demo

Custom RAG flows

- Design your own RAG flow
- Dependency
  - spring-ai-rag

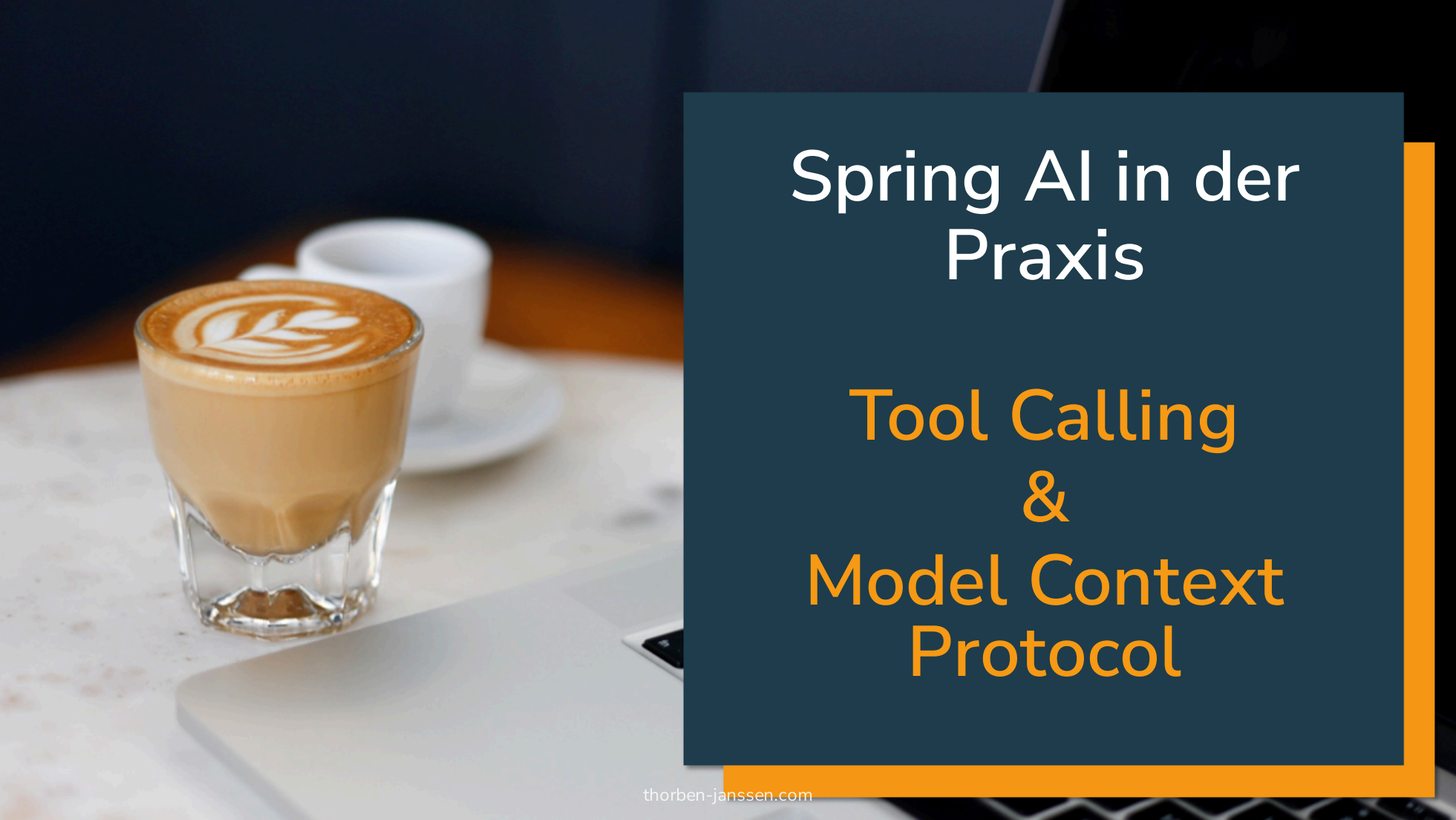
Custom RAG flows

- Pre-Retrieval
  - Optional query transformation
- Retrieval
- Post-Retrieval
  - Optional document post-processing
  - Prompt augmentation
- Generation / LLM response

Live Demo

Questions





Spring AI in der Praxis

Tool Calling & Model Context Protocol

Tool Calling & MCP

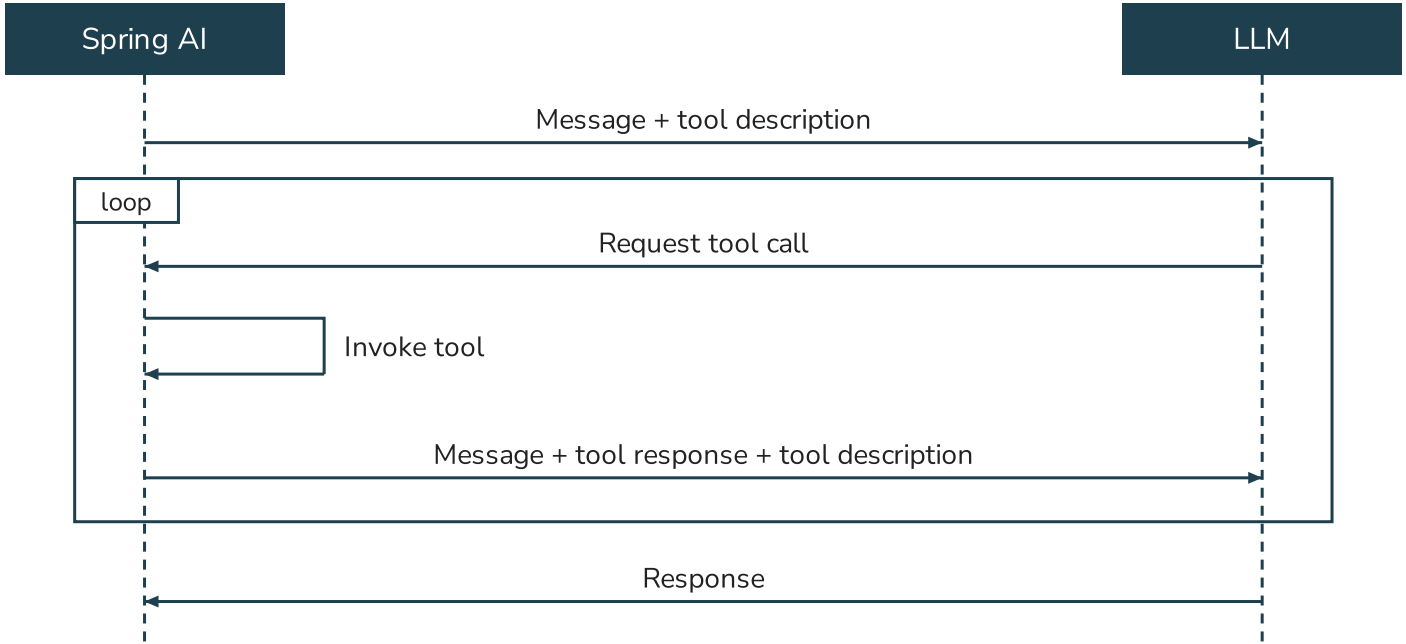
- Provide access to custom APIs
  - Request information
  - Trigger operations

Tool Calling



Tool Calling / Function Calling

- Give model access to APIs



Live Demo

Manage Tools

- Problem
  - Tool definitions increase context size

Tool Search Tool

- Tell LLM to search for tools
  - Based on Advisor
- Part of Spring AI since V 2.0.0
  - previously Spring AI Community project
    - https://github.com/spring-ai-community/spring-ai-tool-search-tool

Live Demo

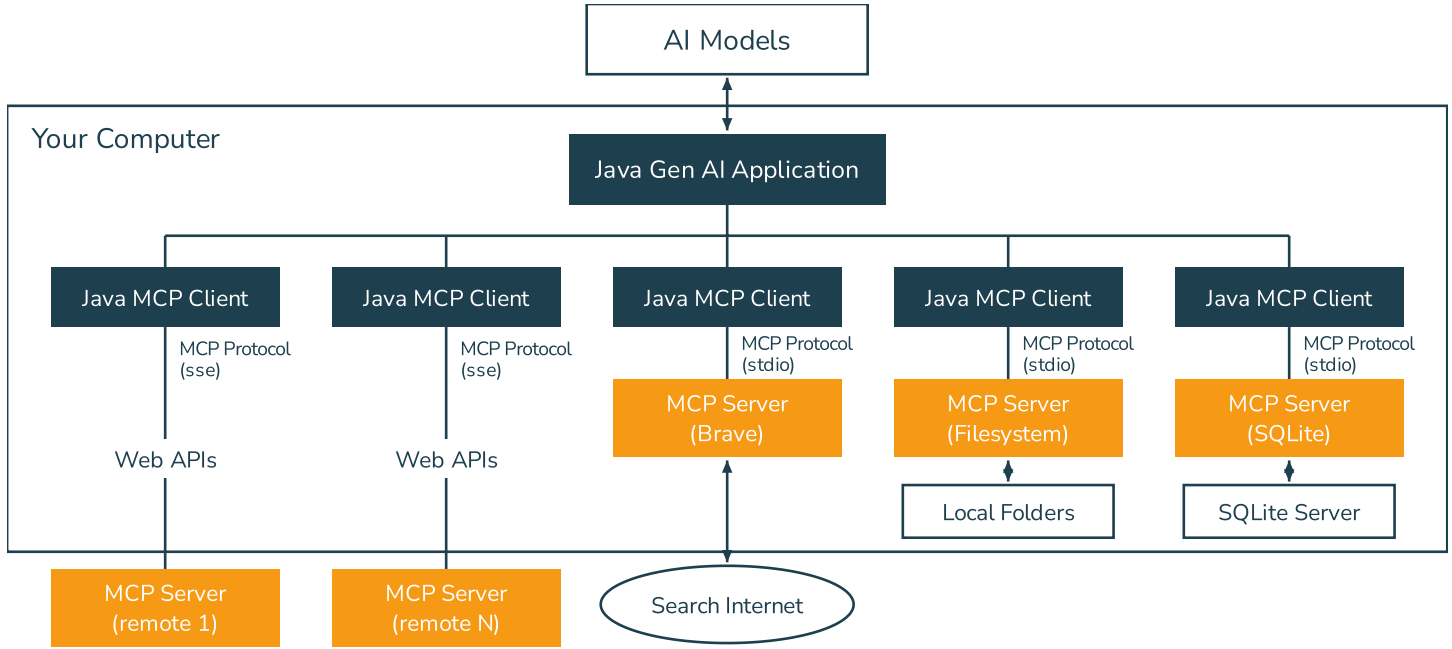
Model Context Protocol (MCP)



MCP

- = tool calling for external services
- Standardized protocol
  - https://modelcontextprotocol.io/

MCP



- Based on : https://docs.spring.io/spring-ai/reference/api/mcp/mcp-overview.html

Dependencies

- Client
  - spring-ai-starter-mcp-client
  - spring-ai-starter-mcp-client-webflux
- Server
  - spring-ai-starter-mcp-server
  - spring-ai-starter-mcp-server-webmvc

MCP Annotations

Server	
`@McpTool`	tool call
`@McpResource`	access to a resource via URI
`@McpPrompt`	generates a prompt
`@McpComplete`	auto-completion

Live Demo

MCP Annotations

Client

`@McpLogging`	handles log messages
`@McpSampling`	handles sampling request
`@McpElicitation`	handles elicitation request
`@McpProgress`	handles progress notifications

MCP Annotations

Client

`@McpToolListChange`	handles tool list change
`@McpResourceListChange`	handles res. list change
`@McpPromptListChange`	handles prompt list change

Security



MCP Security

- Spring AI Community project
- Based on Spring Security
  - Oauth 2.0
  - API key-based

MCP Security

- Documentation
  - https://docs.spring.io/spring-ai/reference/2.0/api/mcp/mcp-security.html
- Github
  - https://github.com/spring-ai-community/mcp-security

Questions

